This implementation plan integrates your serialised GPR surrogate models with NSGA-II multi-objective optimisation, Bayesian optimisation via Expected Improvement, and an experimental validation protocol — each grounded in current LMD/DED literature. The plan is structured as five sequential phases, with the GPR-NSGA-II pipeline as the critical core contribution.

Architecture Overview

Your completed notebook produces serialised `.pkl` GPR and Multi-Output GPR (MOGPR) surrogates that map process parameters (Plaser,vscan,m˙powder)(P_{laser}, v_{scan}, \ dot{m}_{powder})(Plaser,vscan,m˙powder) to bead geometry responses (height error, aspect ratio deviation). The next phase inverts this relationship: rather than predicting outputs *from* parameters, you use a genetic algorithm to search the parameter space for combinations that optimise multiple geometric objectives simultaneously. This surrogate-embedded optimisation pipeline is precisely the AIDED framework architecture described by Shang et al., where ML models and a genetic algorithm are combined to "inversely identify optimal process parameters directly from customisable application objectives." The key distinction of your approach is using GPR specifically, which provides posterior uncertainty — enabling not just optimisation, but *uncertainty-aware* optimisation through Expected Improvement.[ppl-ai-file-upload.s3.amazonaws] [pmc.ncbi.nlm.nih]

Phase 1: GPR-Surrogate NSGA-II

Theoretical Justification

NSGA-II (Deb et al., 2002) is the standard algorithm for this application. It applies non-dominated sorting and crowding distance to simultaneously find a Pareto front across competing objectives. In the LMD/laser cladding literature, NSGA-II has been repeatedly validated as the appropriate solver when a surrogate model provides the fitness evaluation: Fan et al. used SSA-BP + NSGA-II to optimise dilution rate, hardness, and impact energy; Dai et al. used GA-GBDT + NSGA-II to minimise porosity and maximise microhardness of RHEA coatings; and most directly, a 2025 study on laser-arc hybrid additive manufacturing demonstrated GPR specifically combined with NSGA-II to minimise geometric deviation and maximise deposition energy utilisation, confirming GPR's suitability as the NSGA-II fitness oracle.[pmc.ncbi.nlm.nih]Predictive-Modeling-and-Optimization-of-Layer-Cladded-Ti-Al-Nb-High-Entropy-Alloys-Using-Machine.pdf+1

The implementation uses `pymoo`, the Python multi-objective optimisation library used by Shang et al. in AIDED, which provides a clean `Problem` wrapper that accepts callable objective functions — making surrogate embedding straightforward.[anyoptimization][ppl-ai-file-upload.s3.amazonaws]

Implementation

Step 1 — Problem Definition. Wrap your loaded GPR models inside a `pymoo.Problem` subclass. The decision variables are your three process parameters bounded by your experimental

window; the two objectives are height error $|H_{pred} - H_{target}|$ $|H_{pred} - H_{target}|$ and aspect ratio deviation $|\frac{W}{H} - AR_{target}|$ $|HW - ARtarget|$:

```
python
import numpy as np
import pickle
from pymoo.core.problem import Problem
from pymoo.algorithms.moo.nsga2 import NSGA2
from pymoo.termination import get_termination
from pymoo.optimize import minimize

# Load serialised GPR surrogates
with open("gpr_height.pkl", "rb") as f:
    gpr_height = pickle.load(f)
with open("gpr_width.pkl", "rb") as f:
    gpr_width = pickle.load(f)

# Target geometry (set from application requirements)
H_target = 1.0   # mm
AR_target = 2.5  # Width/Height

class LMDOptimisationProblem(Problem):
    """
    Decision variables: [laser_power, scan_speed, powder_feed_rate]
    Objectives: minimise height_error, minimise AR_deviation
    """
    def __init__(self):
        # Bounds from experimental design space
        super().__init__(
            n_var=3,
            n_obj=2,
            n_constr=0,
            xl=np.array([200.0, 4.0, 2.0]),   # [W, mm/s, g/min]
            xu=np.array([600.0, 12.0, 8.0])
        )

    def _evaluate(self, X, out, *args, **kwargs):
        H_pred  = gpr_height.predict(X)
        W_pred  = gpr_width.predict(X)
        AR_pred = W_pred / np.maximum(H_pred, 1e-6)

        f1 = np.abs(H_pred  - H_target)       # height error
        f2 = np.abs(AR_pred - AR_target)      # aspect ratio deviation
```

```
        out["F"] = np.column_stack([f1, f2])
```

Step 2 — Run NSGA-II. With population size of 100 and 200 generations (consistent with parameters in Fan et al. and Dai et al.):Predictive-Modeling-and-Optimization-of-Layer-Cladded-Ti-Al-Nb-High-Entropy-Alloys-Using-Machine.pdf+1

```
python
problem   = LMDOptimisationProblem()
algorithm = NSGA2(pop_size=100, eliminate_duplicates=True)
termination = get_termination("n_gen", 200)

result = minimize(
    problem,
    algorithm,
    termination,
    seed=42,
    save_history=True,
    verbose=True
)

# Extract Pareto-optimal solutions
pareto_X = result.X    # Pareto-optimal process parameters
pareto_F = result.F    # Corresponding objective values
```

Step 3 — Use MOGPR for joint uncertainty quantification. Repeat with the Multi-Output GPR to get correlated height/width uncertainty estimates simultaneously:

```
python
with open("mogpr.pkl", "rb") as f:
    mogpr = pickle.load(f)

# Returns mean predictions and covariance matrix per sample
mogpr_preds_mean, mogpr_preds_std = mogpr.predict(pareto_X, return_std=True)
```

Step 4 — Pareto Front Visualisation and Knee-Point Selection. Plot the Pareto front and identify the "knee point" — the solution offering the best trade-off — using hypervolume or the min-distance to the ideal point method recommended in recent surrogate-based optimisation literature. [pmc.ncbi.nlm.nih]

```
python
from pymoo.decomposition.asf import ASF
import matplotlib.pyplot as plt

# Knee-point via Achievement Scalarisation Function (ASF)
weights = np.array([0.5, 0.5])
decomp  = ASF()
```

```
knee_idx = decomp.do(pareto_F, 1 / weights).argmin()
knee_params = pareto_X[knee_idx]

print(f"Knee-point parameters: P={knee_params[0]:.1f}W, "
      f"v={knee_params[1]:.2f}mm/s, ṁ={knee_params[2]:.2f}g/min")

plt.scatter(pareto_F[:,0], pareto_F[:,1], c='steelblue', label='Pareto Front')
plt.scatter(pareto_F[knee_idx,0], pareto_F[knee_idx,1],
            c='red', s=100, zorder=5, label='Knee Point')
plt.xlabel('Height Error (mm)')
plt.ylabel('Aspect Ratio Deviation')
plt.legend(); plt.tight_layout(); plt.savefig('pareto_front.png')
```

For surrogate-assisted variants that reduce the number of expensive function evaluations, the `pysamoo` library's `SSA-NSGA2` is directly available and uses the GPR as a surrogate to propose infill candidates without evaluating real experiments at every generation.[anyoptimization]

Phase 2: Uncertainty-Aware Bayesian Optimisation

Theoretical Justification

GPR's posterior variance is its primary advantage over competing surrogates for this application. The Expected Improvement (EI) acquisition function exploits this to balance exploration of uncertain regions with exploitation of known good regions — exactly the behaviour needed when N=89 may be insufficient to fully characterise the parameter space. Bayesian optimisation with GPR has been applied directly to EHLA (Extreme High-Speed Laser Material Deposition), demonstrating that BO "effectively accelerates process development" and "achieves higher layer thicknesses with fewer experiments" than grid-search approaches — a critical advantage given the cost of LMD experiments.[publications.rwth-aachen]

Implementation

The EI acquisition function is:

EI(x)=(μ(x)−f ∗ −ξ)Φ(Z)+σ(x)ϕ(Z)\text{EI}(\mathbf{x}) = (\mu(\mathbf{x}) - f^* - \xi) \Phi(Z) + \sigma(\mathbf{x})\phi(Z)EI(x)=(μ(x)−f ∗ −ξ)Φ(Z)+σ(x)ϕ(Z)

where Z=μ(x)−f ∗ −ξσ(x)Z = \frac{\mu(\mathbf{x}) - f^* - \xi}{\sigma(\mathbf{x})}Z=σ(x)μ(x)−f ∗ −ξ, f ∗ f^*f ∗ is the current best observed value, and ξ\xiξ is the exploration–exploitation trade-off parameter.

```
python
from scipy.stats import norm
from scipy.optimize import differential_evolution

def expected_improvement(X, gpr, f_best, xi=0.01):
```

```
    """
    Compute EI for height-error minimisation using the saved GPR.
    X  : (n_candidates, n_features)
    """
    X = np.atleast_2d(X)
    mu, sigma = gpr.predict(X, return_std=True)
    sigma = sigma.reshape(-1, 1)
    mu    = mu.reshape(-1, 1)

    improvement = f_best - mu - xi
    Z   = improvement / (sigma + 1e-9)
    ei  = improvement * norm.cdf(Z) + sigma * norm.pdf(Z)
    ei[sigma < 1e-10] = 0.0
    return ei.flatten()

# Find the next most informative point to experimentally evaluate
def next_sample(gpr, X_observed, f_best, bounds):
    def neg_ei(x):
        return -expected_improvement(
            x.reshape(1, -1), gpr, f_best
        )
    result = differential_evolution(
        neg_ei, bounds=bounds, seed=42, maxiter=500
    )
    return result.x

# Define bounds: [laser_power, scan_speed, powder_feed_rate]
bounds = [(200, 600), (4, 12), (2, 8)]
x_next = next_sample(gpr_height, X_train, f_best=current_best_height_error, bounds=bounds)
print(f"Next recommended experiment: {x_next}")
```

This turns the process parameter search into a closed-loop active learning cycle: each EI-recommended experiment returns new data, the GPR is retrained, and the acquisition is recalculated. For multi-objective BO, the `botorch` library provides the `qNEHVI` (Noisy Expected Hypervolume Improvement) acquisition function, which directly extends EI to multiple competing objectives.pmc.ncbi.nlm.nih+1

Phase 3: Experimental Validation Protocol

Selection Criteria

From the Pareto front, select **7–10 candidate parameter sets** covering diverse trade-off positions — not just the knee point — to constitute a statistically meaningful validation set. Recommended selection strategy:

Selection Criterion	Purpose	Number of Points
Knee-point (min ASF distance)	Best trade-off solution	1
Min height error	Geometric precision priority	2
Min AR deviation	Form accuracy priority	2
High-uncertainty (max σ_GPR)	Identify model blind spots	2
Random interior Pareto points	Broad coverage	2

This mirrors the validation methodology in Fan et al., where an optimal parameter set plus three control-group parameters were tested, and the AIDED framework, where "the effectiveness of AIDED is experimentally validated by solving a multi-objective optimisation problem."Multi-objective-process-parameters-optimization-of-laser-repairing-steel-in-pressure-vessels-usi.PDF+1

Validation Metrics

For each of the selected parameter sets, compare GPR-predicted vs. measured bead geometry using:

- **Absolute prediction error** per output (mm for height/width)
- **Relative error (%)** — should be within the MAPE range established in your cross-validation (i.e., the GPR's known test-set accuracy)
- **Coverage probability** — check whether measured values fall within the GPR's 95% posterior interval [μ±1.96σ][\mu \pm 1.96\sigma][μ±1.96σ]

Coverage probability is the key uncertainty-aware validation metric; a well-calibrated GPR should achieve ~95% empirical coverage. Document any systematic deviations as "model inadequacy" regions and use them to motivate active learning.[pmc.ncbi.nlm.nih]

Phase 4: SHAP-Informed Process Window Analysis

Your existing SHAP outputs provide the feature importance ranking across the parameter space. The process window analysis proceeds as follows:

Step 1 — Map GPR uncertainty across the design space. Generate a 2D grid over any two process parameters (holding the third fixed at its SHAP-identified most-influential value) and plot the GPR posterior standard deviation σ(x)\sigma(\mathbf{x})σ(x):

```
python
from sklearn.gaussian_process import GaussianProcessRegressor

P_grid = np.linspace(200, 600, 50)
v_grid = np.linspace(4, 12, 50)
```

```
P_mesh, v_mesh = np.meshgrid(P_grid, v_grid)
X_grid  = np.column_stack([
    P_mesh.ravel(),
    v_mesh.ravel(),
    np.full(P_mesh.size, 5.0)  # fix powder_feed at median
])

_, sigma_grid = gpr_height.predict(X_grid, return_std=True)
sigma_map = sigma_grid.reshape(P_mesh.shape)

# Overlay NSGA-II Pareto solutions on the uncertainty map
plt.contourf(P_mesh, v_mesh, sigma_map, levels=20, cmap='viridis')
plt.colorbar(label='GPR Posterior σ (height prediction)')
plt.scatter(pareto_X[:,0], pareto_X[:,1],
            c='red', s=30, label='Pareto Solutions')
plt.xlabel('Laser Power (W)'); plt.ylabel('Scan Speed (mm/s)')
plt.title('Process Window: GPR Uncertainty Map with Pareto Overlay')
```

Step 2 — Cross-reference SHAP importances. Regions where SHAP indicates high sensitivity *and* GPR uncertainty is high are the highest-priority targets for additional experiments — these are areas where (a) process parameters matter most and (b) the model is least reliable. This constitutes the "process window analysis" described in your thesis progression, and directly informs the active learning strategy of Phase 5.[ppl-ai-file-upload.s3.amazonaws]

Phase 5: Active Learning Extension

If your learning curve analysis confirms that CV R^2 is still rising at N=89, the Bayesian Optimisation EI mechanism from Phase 2 doubles as an active learning selector. The key adaptation is to use the **multi-objective EI** (or the uncertainty-weighted criterion) to identify parameter combinations in under-explored corners of the design space, specifically those that are:

1. Far from existing training points in input space (high exploration value)
2. Located within the feasible Pareto region (high exploitation value)
3. Associated with high GPR posterior variance (model uncertainty hotspots)

This three-criterion active learning strategy is consistent with the approach recommended by Shang et al. for extending the AIDED framework: "active learning could be implemented to reduce the amount of data needed in training the ML models." A practical implementation selects the next experiment as:[ppl-ai-file-upload.s3.amazonaws]

$x_* = \text{argmax}_{x \in X} [\lambda_1 \cdot EI(x) + \lambda_2 \cdot \sigma(x) + \lambda_3 \cdot d(x, X_{train})]$ $\mathbf{x}^* = \arg\max_{\mathbf{x} \in \mathcal{X}} \left[\lambda_1 \cdot \text{EI}(\mathbf{x}) + \lambda_2 \cdot \sigma(\mathbf{x}) + \lambda_3 \cdot d(\mathbf{x}, \mathcal{X}_{train}) \right]$ $x_* = \text{argmax}_{x \in X} [\lambda_1 \cdot EI(x) + \lambda_2 \cdot \sigma(x) + \lambda_3 \cdot d(x, X_{train})]$

where λ1,λ2,λ3\lambda_1, \lambda_2, \lambda_3λ1,λ2,λ3 are weighting coefficients and d(⋅)d(\cdot)d(⋅) is the minimum distance to the current training set. Even 5–10 targeted new experiments selected this way are expected to yield a more significant R^2 improvement than 20–30 random additional experiments.

Recommended Implementation Sequence

```
text
Week 1–2:  Phase 1 — GPR-NSGA-II pipeline, Pareto front generation
Week 3:    Phase 4 — Process window / SHAP uncertainty mapping
Week 3–4:  Phase 2 — EI acquisition function, next-experiment recommendations
Week 4–5:  Phase 3 — Physical experiments (7–10 validation points)
Week 6:    Phase 3 — Predicted vs. actual comparison, coverage analysis
Week 7:    Phase 5 — Active learning (if learning curve warrants new data)
Week 8:    Thesis writing — Validation chapter
```

The canonical toolchain is: **scikit-learn** (GPR predict + EI) → **pymoo** (NSGA-II) → **pysamoo** (SSA-NSGA-II surrogate assistance) → **matplotlib/seaborn** (Pareto and process window visualisation) — all Python-native and compatible with your existing notebook infrastructure. [anyoptimization]